orthodox.net Platform Overview

St. Nicholas Orthodox Church

Technical Architecture Summary

March 2026

Abstract

This document provides a high-level technical overview of the orthodox.net platform — the website of St. Nicholas Orthodox Church in McKinney, Texas. The platform consists of two repositories: a quasi-static frontend (`onet-v2`) deployed on a LAMP stack, and a Python Flask backend API (`onet-flask-backend`) deployed on Render.com. This overview describes how the components fit together, summarizes the technology stack and external service integrations, and provides pointers to the detailed technical references for each component. It is intended for stakeholders, new team members, or anyone needing a quick understanding of the system without implementation-level detail.

Contents

1 Introduction

1.1 Background

The original orthodox.net website was built in the early 2000s as a traditional LAMP stack (Linux, Apache, MySQL, PHP) site. Over two decades it accumulated thousands of static HTML pages — sermons, articles, scripture readings, catechism materials, saints' lives, and more — many authored in Microsoft Word and exported to HTML.

The **onet-v2 project** was initiated to modernize the site while keeping it on the same hosting provider (Inoa/Orthodox Internet Services) and progressively replacing legacy pages without modifying them.

1.2 Repositories

The platform spans two GitHub repositories:

Repository	Purpose
`onet-v2` https://github.com/GZancewicz/onet-v2	Frontend: HTML, CSS, JavaScript, PHP fetcher scripts, Apache configuration, Python utilities. Deployed on Inoa shared hosting.
`onet-flask-backend` https://github.com/GZancewicz/onet-flask-backend	Backend API: Python Flask REST API aggregating content from Ghost CMS, YouTube, and Google Calendar. Deployed on Render.com.

1.3 Detailed References

Each repository has its own in-depth technical reference document:

- **Frontend**: `onet-v2/doc/latex/onet_frontend/onet_frontend.pdf` (36 pages) — covers production server architecture, Apache routing, legacy migration system, JavaScript architecture, deployment, and troubleshooting.
- **Backend**: `onet-flask-backend/doc/latex/onet_backend/onet_backend.pdf` — covers API endpoints, database schema, caching strategy, Render.com deployment, and module internals.

This overview document intentionally avoids duplicating their content. It focuses on how the pieces fit together.

2 System Architecture

Figure 1 shows the complete platform architecture.

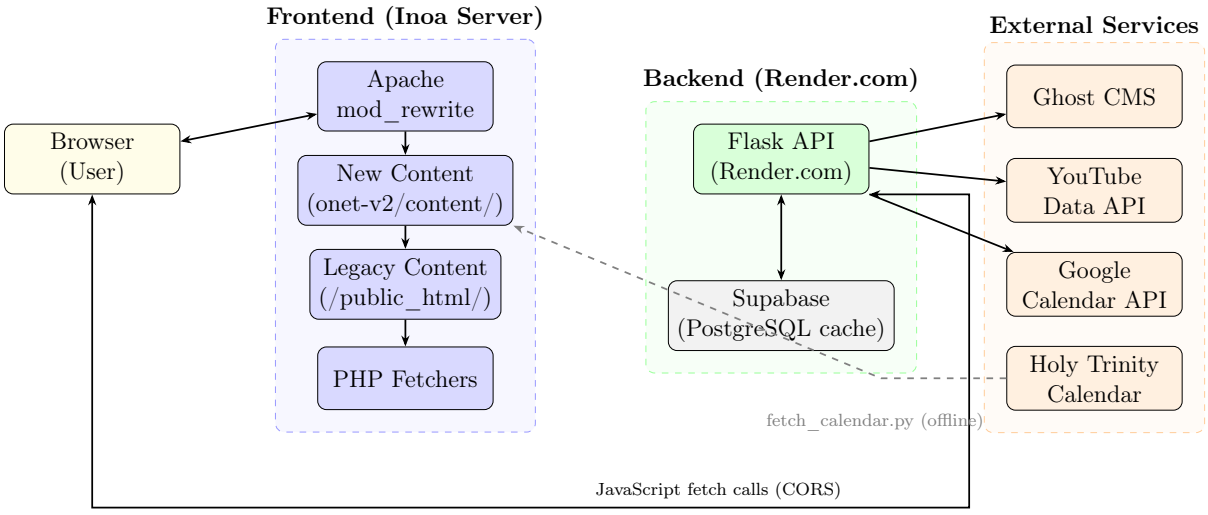

Figure 1: Complete orthodox.net platform architecture.

2.1 How the Components Interact

1. The **browser** makes HTTP requests to `orthodox.net`, which is served by Apache on the Inoa shared hosting server.

2. Apache's `.htaccess` routing checks for new content in `/new/onet-v2/content/` first, falling back to legacy `/public_html/` pages.

3. For legacy content categories (sermons, articles, etc.), Apache rewrites the URL to a **shell HTML page** that uses JavaScript to fetch content from a **PHP fetcher script**, which reads and cleans the original legacy HTML file.

4. For dynamic content (blog posts, videos, calendar), JavaScript in the browser makes direct **fetch calls to the Flask API** on Render.com (cross-origin, CORS-enabled).

5. The Flask API aggregates content from **Ghost CMS** (articles), **YouTube** (videos), and **Google Calendar** (services schedule), caching responses in **Supabase** (PostgreSQL) to reduce API quota usage.

6. The "Today's Saints & Readings" section uses **pre-generated static HTML files** scraped from Holy Trinity Orthodox Church's calendar. These are generated offline by a Python script and deployed via git.

3 Technology Stack

3.1 Frontend Stack

Technology	Usage
HTML5, CSS3	Markup and styling (Skeleton grid framework, Alegreya font)
Vanilla JavaScript	Client-side logic; no frameworks, no build process
Apache 2.x	Web server with `mod_rewrite` for URL routing

PHP	Minimal: content fetcher scripts for legacy migration
Python 3	Utility scripts (template updates, calendar fetching, image conversion)
Git	Version control; deployed via `git pull` on production

3.2 Backend Stack

Technology	**Usage**
Python 3.12	Core application language
Flask 2.1	Web framework for REST API
Gunicorn	WSGI server for production
Supabase	PostgreSQL caching layer (calendar events, YouTube videos)
Flasgger	Auto-generated Swagger API documentation
Render.com	Cloud hosting (staging + production environments)

4 External Service Integrations

Service	**Consumed By**	**Purpose**
Ghost CMS	Backend API	Headless CMS for articles and blog posts
YouTube Data API	Backend API	Video content from `@orthodoxnet` channel
Google Calendar	Backend API	Church services and events schedule
Holy Trinity Orthodox	Frontend (offline)	Saints and readings calendar data (pre-generated)
Google Analytics	Frontend	Site usage analytics
PayPal SDK	Frontend	Donation processing
Inoa/OIS	Frontend	Production LAMP hosting (cPanel)
Render.com	Backend API	Cloud hosting (staging + production)
Supabase	Backend API	PostgreSQL caching layer

5 Frontend Summary

The frontend (`onet-v2`) is a quasi-static website served from a LAMP stack. Key architectural features:

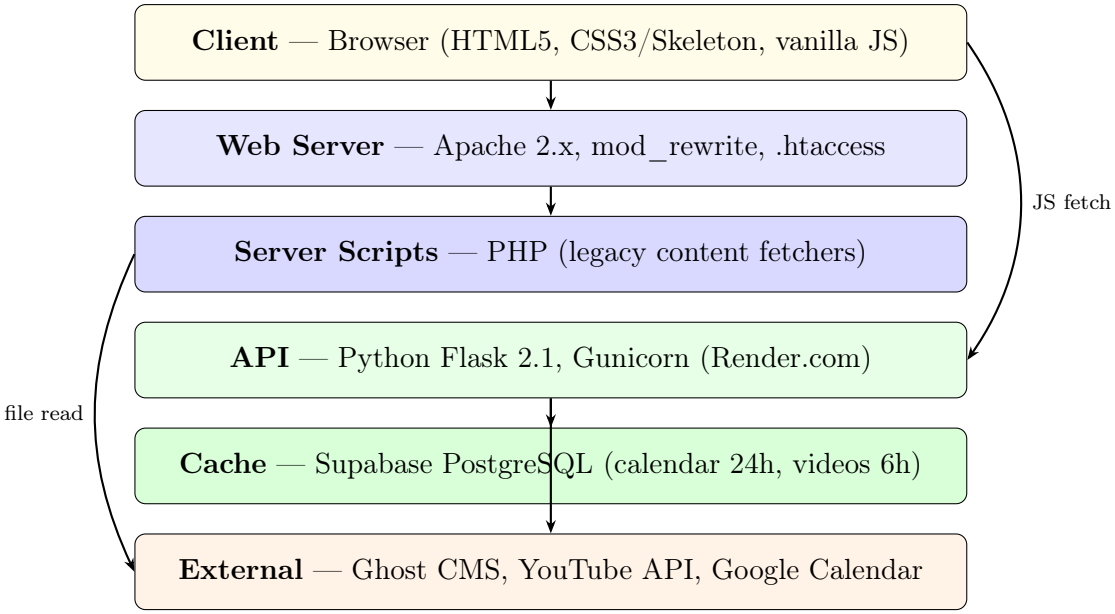

Figure 2: Technology stack from client to external data sources.

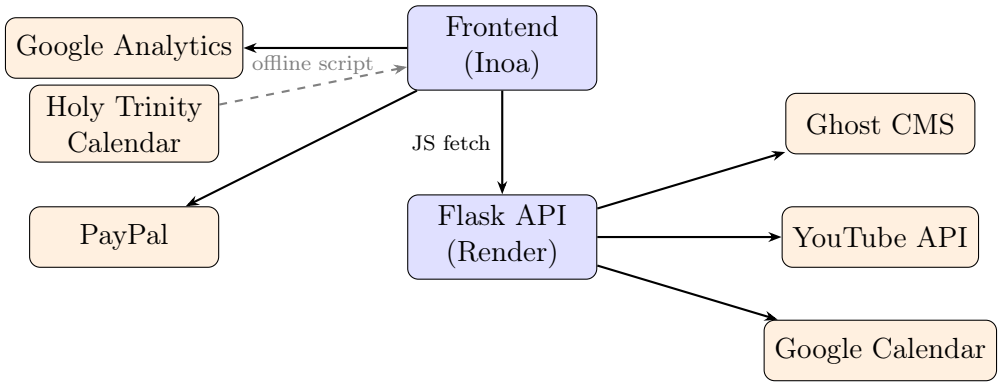

Figure 3: External service integrations across both components.

- **Progressive migration**: New pages in `/new/onet-v2/content/` coexist with legacy pages in `/public_html/`. Apache `.htaccess` rules check new content first and fall back to legacy.
- **Legacy content migration**: 24 content categories are served through shell pages that dynamically load, clean, and re-template legacy HTML via PHP fetcher scripts and JavaScript.
- **No build process**: All files are served directly — vanilla HTML, CSS, and JavaScript with query-string cache busting.
- **Dynamic content**: Homepage sections (videos, services schedule, blog posts) are populated by JavaScript calling the Flask backend API.
- **Saints & Readings**: Pre-generated static HTML files from Holy Trinity Orthodox Church, loaded client-side by `synaxarion.js`. Must be regenerated every 90 days.
- **Deployment**: `git pull` on the production server via cPanel terminal. The production `.htaccess` is maintained manually on the server (not deployed via git).

For full details, see the frontend technical reference (`onet_frontend.pdf`).

6 Backend API Summary

The backend (`onet-flask-backend`) is a Python Flask REST API that aggregates content from external services. Key architectural features:

- **Single Flask app**: All routes defined in `app.py`, with integration logic in separate modules (`fetch_ghost.py`, `fetch_youtube.py`, `fetch_calendar.py`).
- **Caching**: Supabase (PostgreSQL) caches calendar events (24-hour TTL) and YouTube videos (6-hour TTL) to reduce API quota usage. Cache misses trigger live API calls with automatic upsert.
- **API documentation**: Interactive Swagger UI at `/docs` endpoint, auto-generated by Flasgger.
- **Deployment**: Two Render.com Web Services — production (from `main` branch) and staging (from `staging` branch). Auto-deploy on push.
- **CORS**: Open to all origins (`*`), enabling direct browser-to-API calls from the frontend.
- **No authentication**: All endpoints are public. Admin cache management endpoints are also unauthenticated (known technical debt).

6.1 API Endpoint Summary

Endpoint Group	Key Endpoints
Articles (Ghost CMS)	`/onet_article_data`, `/onet_article`, `/article_by_slug`, `/parish_life`, `/motd`, `/legacy_article`
Videos (YouTube)	`/latest_videos` (cached), `/homilies_videos`, `/childrens_videos`, `/catechism_videos`
Calendar (Google)	`/schedule` (cached, next 10 days, US/Central)
System	`/heartbeat`, `/docs` (Swagger UI)
Admin	`POST /cache_videos`, `/view_video_cache`

For full endpoint documentation with request/response examples, see the backend technical reference (`onet_backend.pdf`).

7 Deployment Overview

Component	Hosting	Deployment Method

Frontend	Inoa/OIS (cPanel)	Manual `git pull` via cPanel terminal
Backend (prod)	Render.com	Auto-deploy on push to `main` branch
Backend (staging)	Render.com	Auto-deploy on push to `staging` branch

Important: The frontend's production `.htaccess` at `~/public_html/.htaccess` is *not* deployed via `git pull`. It must be maintained manually on the server. See the frontend technical reference for details.

8 Key URLs

Resource	URL
Live site	https://www.orthodox.net
Frontend repo	https://github.com/GZancewicz/onet-v2
Backend repo	https://github.com/GZancewicz/onet-flask-backend
API (production)	https://production-orthodox-net-flask.onrender.com
API (staging)	https://staging-orthodox-net-flask.onrender.com
API docs (Swagger)	https://production-orthodox-net-flask.onrender.com/docs
API health check	https://production-orthodox-net-flask.onrender.com/heartbeat

9 Credentials and Access

To maintain the platform, you need access to:

1. **GitHub**: Write access to both repositories
2. **Inoa/OIS cPanel**: Hosting control panel (file manager, terminal, PHP config)
3. **Render.com**: Dashboard for backend deployment (environment variables, logs, manual deploys)
4. **Ghost CMS**: Admin login for article/post management
5. **Google Cloud**: API keys for YouTube Data API and Google Calendar API
6. **Supabase**: Project dashboard for database management
7. **Google Analytics**: Access to property `G-D9YEGT19W7`
8. **PayPal**: Business account for donation configuration

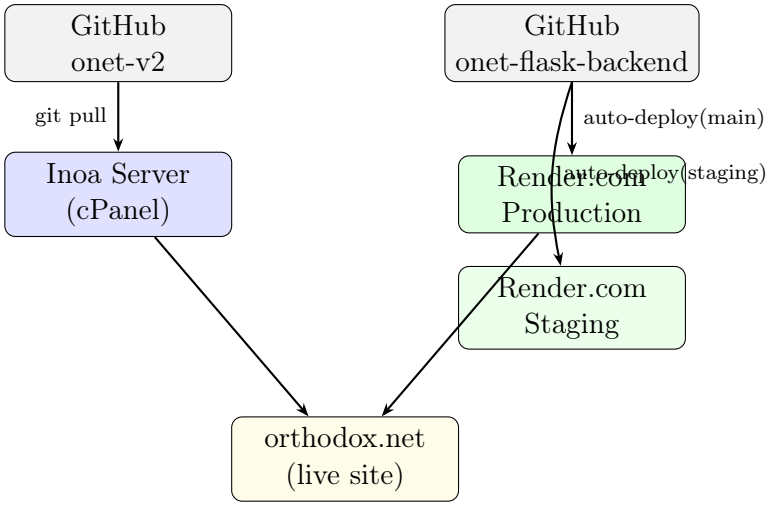

Figure 4: Deployment pipeline for both components.

9. **DNS**: Managed by Inoa/OIS

Known issue: Ghost CMS API key and YouTube API key are currently hardcoded in the backend source code rather than read from environment variables. See the backend technical reference for details on this technical debt.

10 Periodic Maintenance

Task	**Frequency**	**Details**
Calendar data refresh	Every 90 days	Run `fetch_calendar.py` to regenerate saints/readings HTML files. Run locally and on server.
Backend monitoring	As needed	Check `/heartbeat`. Render free-tier services may sleep after inactivity.
API quota monitoring	Monthly	YouTube Data API has daily quota limits. Caching reduces usage but monitor via Google Cloud console.
SSL certificates	Automatic	Inoa and Render both handle SSL renewal automatically.
Production .htaccess	On routing changes	Must be updated manually on server when adding new legacy folder conversions.